



S. B. JAIN INSTITUTE OF TECHNOLOGY, MANAGEMENT & RESEARCH, NAGPUR.

POST LAB

Aim: To write and execute a **menu-driven C program** to simulate different **Page Replacement Algorithms** such as **FIFO (First In First Out)**, **LRU (Least Recently Used)**, and **Optimal**, and to calculate the **number of page faults** for a given page reference string.

Name of Student: **Shweta Bambode**

USN No : **CD24052**

Semester/Year: **4th / 2nd**

Academic Session: **2025-26**

Date of Performance:

Date of Performance:

AIM:

To write and execute a **menu-driven C program** to simulate different **Page Replacement Algorithms** such as **FIFO (First In First Out)**, **LRU (Least Recently Used)**, and **Optimal**, and to calculate the **number of page faults** for a given page reference string.

OBJECTIVES:

1. To understand the concept of **memory management in operating systems**.
2. To study the **working of page replacement algorithms**.
3. To implement **FIFO, LRU, and Optimal page replacement algorithms using C programming**.
4. To observe and calculate the **number of page faults** for a given page reference string.
5. To compare the performance of **different page replacement algorithms**.

THEORY:

Memory management is an important function of an **Operating System (OS)**. It manages how memory is allocated and used by different programs. When programs are executed, they are divided into small parts called **pages**. The main memory is divided into fixed-size blocks called **frames**.

Sometimes the required page is not present in the memory. In this situation, the operating system must bring that page from secondary storage to the main memory. This situation is called a **page fault**.

If all frames in the memory are already full, the operating system must remove one page from memory to make space for the new page. This process is known as **Page Replacement**. Different algorithms are used to decide which page should be removed. These algorithms help in reducing the number of page faults and improving system performance.

Important Points

- **Page:** A fixed-size block of a process stored in memory.
- **Frame:** A fixed-size block of physical memory where pages are loaded.
- **Page Fault:** It occurs when the required page is not available in the main memory.
- **Page Replacement:** The process of replacing an existing page in memory with a new page when frames are full.

Types of Page Replacement Algorithms

1. FIFO (First In First Out)

- The page that enters memory first is removed first.
- It works like a queue structure.
- It is simple and easy to implement.

2. LRU (Least Recently Used)

- The page that has not been used for the longest time is replaced.
- It keeps track of page usage.
- It generally performs better than FIFO.

3. Optimal Page Replacement

- The page that will not be used for the longest time in the future is replaced.
- It gives the minimum number of page faults.
- It is mostly used for comparison in studies because future knowledge is required.

Page Replacement Simulator

A **Page Replacement Simulator** program is used to demonstrate how different page replacement algorithms work. It accepts the page reference string and number of frames as input and simulates the algorithms. The program then calculates the number of page faults produced by each algorithm.

This helps students understand the behavior and efficiency of different memory management techniques used in operating systems.

CODE :

```
Shweta Bamcode@Shweta MSYS ~  
$ nano PRS.c
```

```
GNU nano 8.7  
#include <stdio.h>  
  
int frames[10], pages[50], n, f;  
  
void fifo()  
{  
    int i, j, pos = 0, faults = 0, found;  
  
    for(i = 0; i < f; i++)  
        frames[i] = -1;  
  
    for(i = 0; i < n; i++)  
    {  
        found = 0;  
  
        for(j = 0; j < f; j++)  
        {  
            if(frames[j] == pages[i])  
            {  
                found = 1;  
                break;  
            }  
        }  
  
        if(found == 0)  
        {  
            frames[pos] = pages[i];  
            pos = (pos + 1) % f;  
            faults++;  
        }  
    }  
  
    printf("\nPage Faults (FIFO) = %d\n", faults);  
}
```



GNU nano 8.7

```
void lru()
{
    int i, j, faults = 0, found, least, index;
    int time[10], counter = 0;

    for(i = 0; i < f; i++)
    {
        frames[i] = -1;
        time[i] = 0;
    }

    for(i = 0; i < n; i++)
    {
        found = 0;
        for(j = 0; j < f; j++)
        {
            if(frames[j] == pages[i])
            {
                counter++;
                time[j] = counter;
                found = 1;
                break;
            }
        }
        if(found == 0)
        {
            least = time[0];
            index = 0;
            for(j = 1; j < f; j++)
            {
                if(time[j] < least)
                {
                    least = time[j];

```



GNU nano 8.7

```
                {
                    least = time[j];
                    index = j;
                }

            frames[index] = pages[i];
            counter++;
            time[index] = counter;
            faults++;
        }
    }

    printf("\nPage Faults (LRU) = %d\n", faults);
}

void optimal()
{
    int i, j, k, pos, faults = 0, found;
    int farthest, index;

    for(i = 0; i < f; i++)
        frames[i] = -1;

    for(i = 0; i < n; i++)
    {
        found = 0;
        for(j = 0; j < f; j++)
        {
            if(frames[j] == pages[i])
            {
                found = 1;
                break;
            }
        }
    }
}
```

```
GNU nano 8.7

    if(found == 0)
    {
        pos = -1;
        farthest = i + 1;

        for(j = 0; j < f; j++)
        {
            for(k = i + 1; k < n; k++)
            {
                if(frames[j] == pages[k])
                {
                    if(k > farthest)
                    {
                        farthest = k;
                        pos = j;
                    }
                    break;
                }
            }

            if(k == n)
            {
                pos = j;
                break;
            }
        }

        if(pos == -1)
            pos = 0;

        frames[pos] = pages[i];
        faults++;
    }
}
```

```
GNU nano 8.7
}

printf("\nPage Faults (Optimal) = %d\n", faults);
}

int main()
{
    int i, choice;

    printf("Enter number of pages: ");
    scanf("%d", &n);

    printf("Enter page reference string:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &pages[i]);

    printf("Enter number of frames: ");
    scanf("%d", &f);

    do
    {
        printf("\n1. FIFO\n2. LRU\n3. Optimal\n4. Exit");
        printf("\nEnter your choice: ");
        scanf("%d", &choice);

        switch(choice)
        {
            case 1: fifo(); break;
            case 2: lru(); break;
            case 3: optimal(); break;
        }

    } while(choice != 4);

    return 0;
}
```

OUTPUT:

```
M ~
$ nano PRS.c

Shweta Bambode@Shweta MSYS ~
$ gcc PRS.c -o PRS

Shweta Bambode@Shweta MSYS ~
$ ./PRS
Enter number of pages: 7
Enter page reference string:
1 2 3 4 1 2 5
Enter number of frames: 3

1. FIFO
2. LRU
3. Optimal
4. Exit
Enter your choice: 1

Page Faults (FIFO) = 7

1. FIFO
2. LRU
3. Optimal
4. Exit
Enter your choice: 2

Page Faults (LRU) = 7

1. FIFO
2. LRU
3. Optimal
4. Exit
Enter your choice: 3

Page Faults (Optimal) = 5
```

CONCLUSION:

The Page Replacement Simulator program was successfully implemented using the C programming language. The program demonstrates the working of FIFO, LRU, and Optimal page replacement algorithms. By running the program with a given page reference string and number of frames, the number of page faults can be calculated. This experiment helps in understanding how operating systems manage memory efficiently and how different page replacement algorithms affect system performance.